

Scope delle variabili

Scope delle variabili

Dove esistono le variabili e come evitare confusione tra funzioni e codice esterno.

Cosa significa scope

Lo **scope** è la zona in cui una variabile esiste ed è leggibile.

Una variabile creata dentro una funzione non è automaticamente disponibile fuori. Una variabile creata fuori non entra automaticamente dentro.

Variabili locali

```
<?php
function saluta() {
    $nome = "Luca";
    echo $nome;
}

saluta();
```

`$nome` esiste solo dentro la funzione `saluta`.

Questo codice invece non funziona come potresti aspettarti:

```
<?php
function preparaNome() {
    $nome = "Luca";
}

preparaNome();
echo $nome;
```

Fuori dalla funzione, `$nome` non esiste.

Parametri

Il modo più pulito per portare dati dentro una funzione e usare parametri.

```
<?php
function saluta($nome) {
    echo "Ciao $nome";
}

saluta("Sara");
```

Valori di ritorno

Il modo più pulito per portare dati fuori da una funzione e usare `return`.

```
<?php
function creaMessaggio($nome) {
    return "Ciao $nome";
}

$messaggio = creaMessaggio("Marta");
echo $messaggio;
```

Variabili globali

PHP permette di usare `global`, ma è meglio limitarlo.

```
<?php
$iva = 22;

function mostraIva() {
    global $iva;
    echo $iva;
}
```

Il codice funziona, ma la funzione dipende da qualcosa che sta fuori. Con parametri e `return`, il collegamento è più chiaro.

Perché lo scope aiuta

Lo scope evita che una variabile venga modificata per sbaglio da una parte lontana del programma. Una funzione con parametri chiari è più prevedibile.

```
<?php
function applicaSconto($prezzo, $sconto) {
    return $prezzo - $sconto;
}

$totale = applicaSconto(30, 5);
echo $totale;
```

La funzione lavora solo con i valori che riceve. Non deve cercare variabili nascoste nel resto del file.

Stesso nome, scope diverso

Puoi avere variabili con lo stesso nome in scope diversi.

```
<?php
$nome = "Luca";

function saluta() {
    $nome = "Sara";
    echo $nome;
}

saluta();
echo $nome;
```

Dentro la funzione `$nome` vale `Sara` . Fuori vale ancora `Luca` .

Regola pratica

Quando scrivi una funzione, prova a farle ricevere tutto ciò che le serve tramite parametri. Se deve produrre un risultato, usa `return` . Tieni `global` come ultima scelta.